

MedSearch: Medical Term Search Tool

Team Name: Group #12

Team Members: Ruby McCandless, Kaden Lendick, John Watson

GitHub Repo: <https://github.com/RubyM0226/DSA-Project-2-Medical-Term-Checker>

Video Demo:

<https://youtu.be/XS7cME1DfbQ>

Problem

As medical and symptom-term databases grow into hundreds of thousands of entries, it becomes harder to search the entries to find needed information. This project compares two different types of navigating these entries using either exact-match lookup or prefix based autocomplete. We are asking the question: "Given a large database of medical terms, which data structure (trie or hash map) better serves a real-world search tool?"

Motivation

Any system that includes searching a database depends on search speed and flexibility. When we tested our structures with a smaller dataset (less than 100 terms), the difference between the two was unrecognizable. When testing with a real-world size dataset (100,000 terms), it all depends on how you are searching for the term. A trie-based system is practical for autocomplete where a hash-based system excels at exact matches. We want to figure out which structure will serve better as a medical search tool and examine the conditions on how each one outperforms the other.

Features

Our project includes:

- Both the trie and the hashmap successfully loading and storing 100,000+ medical and symptom terms without errors
- A user can enter an exact match and have the interface say if the result was found or not using the custom hash map (menu option 1)
- A user can enter a partial term and receive every matching term using the trie (menu option 2)
- Both structures return consistent results for the same query
- The system runs and reports timed benchmarks comparing the structures on different datasets (1k, 10k, 50k, 100k)
- A simple menu driven CLI that ties the features together for a demo

Data

```

DSA-Project-2-Medical-Term-Checker > benchmark_results.csv
1  N,TrieInsert_ms,HashInsert_ms,TrieSearch_ns_avg,HashSearch_ns_avg,ResultsMatch
2
3  1000,0.295,0.0814,110.977,38.535,YES
4
5  10000,1.6763,0.7518,205.75,63.281,YES
6
7  50000,9.5921,4.7327,485.842,135.563,YES
8
9  100000,20.4359,8.9512,462.392,154.633,YES
10
11

```

Tools

- C++17 (implementation language)
- <chrono> (benchmarking and timing)
- Git/Github (version control and collaboration)
- Google Sheets (documenting benchmarking results)

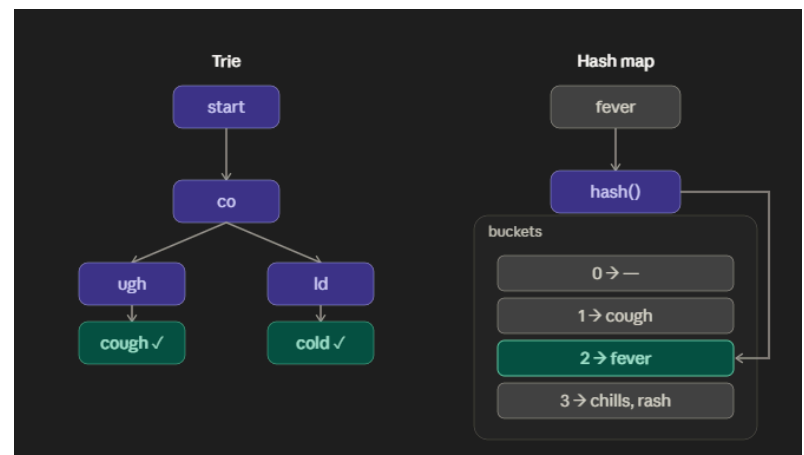
Visuals

Menu-driven CLI interface sketch:

```

-----
Medical Term Search Tool
-----
1. Exact search      (Hash Map)
2. Autocomplete     (Trie)
3. Run benchmarks
4. Exit
-----
Enter choice: _

```



Strategy

We will implement two data structures from scratch and both will use the same dataset of medical terms allowing for a direct comparison.

- Custom Trie: Each node stores a map<char, Node*> of child pointers and a boolean flag marking complete terms. It supports exact search and prefix autocomplete by traversing the relevant subtree and collecting the flagged nodes beneath the prefix.
- Custom Hash Map: A fixed size bucket array with a handwritten hash function with chaining/bucket links. This fixed bucket count avoids rehashing during benchmarks and ensures fair timing across this entire project.

We plan to test across multiple dataset sizes and perform multiple trials. Results will be shown on a data chart.

Additional Data Structures

- `std::map<char, TrieNode*>` — used inside each Trie node to map characters to child nodes
- `std::vector<std::string>` — used to load and pass dataset terms at startup
- `std::vector<HashNode*>` — the underlying bucket array in the Hash Map
- Singly linked list (custom `HashNode*`) — collision chaining within each Hash Map bucket

Distribution of Roles

- Ruby McCandless: Custom Trie implementation (search and autocomplete), menu-driven CLI, write-up, and video presentation
- Kaden Lendick: Custom Hash Map implementation (hash function, chaining, insert, search), benchmark testing module
- John Watson: Data collection, cleaning, and dataset preparation (terms.txt), research

Analysis

No significant changes from the original proposal. Everything remained consistent. The only slight change was how we extracted data. We ended up combining brand name, generic name, dosage, and route into searchable strings to ensure a sufficient number of unique terms.

Big-O Worst Case Time Complexity

Let m = length of the search term, n = total number of terms stored, k = number of autocomplete results returned, and b = number of entries in a single hash bucket.

Trie:

- `insert()`: $O(m)$: walks one node per character, creating nodes as needed. No rebalancing required
- `search()`: $O(m)$: walks one node per character and checks the `isEnd` flag. Terminates early if the path breaks
- `autocomplete()`: $O(m + k)$: $O(m)$ to walk to the prefix node, then $O(k)$ for the subtree collection where k is the total characters across all returned terms

Hash Map:

- `insert()`: $O(m)$ average: $O(m)$ to compute the hash, $O(1)$ to prepend to the bucket chain. Worst case $O(m*n)$ if all terms collide into one bucket
- `search()`: $O(m)$ average: $O(m)$ to compute the hash, $O(1)$ to check the chain head. Worst case $O(m*n)$ under maximum collision
- `autocomplete()`: Not supported

Benchmark Results

Both structures were tested at 1k, 10k, 50k, and 100k terms. All results were averaged over 1,000+ trials per test. The Hash Map outperformed the Trie on both insert and exact-search

speed at every dataset size, running roughly 2-2.5x faster on inserts and approximately 3x faster on exact searches. This aligns with theoretical expectations: the Hash Map computes a single hash and accesses a bucket directly, while the Trie must traverse one node per character. However, the Trie remains the only structure capable of prefix-based autocomplete.

Reflection

Challenges

- Dataset preparation took longer than expected. The openFDA JSON required combining multiple fields to produce enough unique searchable strings to clear 100,000 entries, which was not anticipated at the proposal stage.
- Windows terminal encoding issues caused Unicode characters (dividers, checkmarks, bullets) in the CLI output to render as garbled text in Git Bash, requiring all symbols to be replaced with plain ASCII equivalents.
- Integrating the Trie and Hash Map into a shared main.cpp required agreeing on method signatures and parameter types before either structure was finished, which required early coordination.

What Each Member Learned

- Ruby: Deepened understanding of Trie structure and traversal, particularly how `map<char, Node*>` enables character sets beyond the standard 26-letter alphabet. Also gained experience integrating multiple teammate components into a single working CLI.
- John: I learned that the data structure with the least time complexity isn't always the best solution to achieve efficiency. I also learned the challenge of parsing a massive dataset into a structure that we want.
- Kaden: I mainly learned how to properly structure hash since that's what I mainly worked on as well as doing a lot of like diagnostics the project was very challenging and overall I had a lot of fun just coding and learning new things. I Really work well with teams so this was nothing new, but I had a lot of fun regardless.

References

- U.S. Food and Drug Administration. openFDA Drug NDC Directory.
<https://open.fda.gov/data/downloads/>
- Dhivyesh RK. Diseases and Symptoms Dataset. Kaggle.
<https://www.kaggle.com/datasets/dhivyeshrk/diseases-and-symptoms-dataset>
- Shaffer, C. A. Data Structures and Algorithm Analysis. COP3530 Course Materials, University of Florida.